TP2 : Régression Non linéaire et surapprentissage

Objective

L'objectif de cet exercice est de **comprendre et visualiser le surapprentissage** (*overfitting*) dans les modèles de régression, et d'appliquer des techniques de **validation croisée** et de **régularisation** pour le corriger.

Travail demandé

Étape 1 : Génération et préparation des données

Jeu de données (Fonction sinusoïdale synthétique)

Données générées artificiellement selon : y=sin(1.5x)+0.3x+e

```
np.random.seed(42)
X = np.linspace(-3, 3, 80).reshape(-1, 1)
y_true = np.sin(1.5 * X).ravel() + 0.3 * X.ravel()
y = y_true + np.random.normal(0, 0.3, X.shape[0])

df = pd.DataFrame({"x": X.ravel(), "y": y})
```

Diviser le jeu en :**70 % pour l'entraînement**, **30 % pour le test**.

Étape 2 : Régression polynomiale

Créer des modèles polynomiaux de degré croissant (par exemple de 1 à 15).

Pour chaque modèle :

- Entraîner sur le jeu d'entraînement,
- Calculer la MSE sur l'entraînement et le test.

Tracer les courbes **MSE_train** et **MSE_test** en fonction du degré du polynôme.

Étape 3 : Visualisation des modèles

Tracer, sur le même graphique :

- Les points d'entraînement,
- La vraie fonction y=sin(1.5x)+0.3x
- Les prédictions du modèle pour différents degrés (ex. : 1, 4, 10, 15).

Étape 4 :Validation croisée

Appliquer une **validation croisée (K=5)** pour estimer l'erreur moyenne en fonction du degré du polynôme.

Tracer la **MSE moyenne en validation croisée** en fonction du degré.

Identifier le **degré optimal** qui minimise cette erreur.

Étape 5 : Régularisation Ridge

Choisir un modèle polynomiale de degré élevé (ex. : 10 ou 12).

Entraîner une régression **Ridge** avec plusieurs valeurs du paramètre de régularisation α.

Utiliser **GridSearchCV** pour sélectionner le meilleur α.

Comparer :

- Les MSE d'entraînement et de test avant/après régularisation,
- Les courbes obtenues avec et sans Ridge.